



Red Hat AI

| vLLM



# llm-d Introduction & Status Update

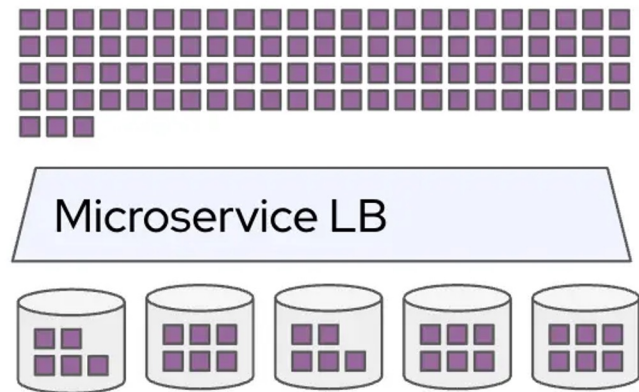
张家驹

红帽大中华区CTO

# Why Distributed Inference?

Modern HTTP requests:

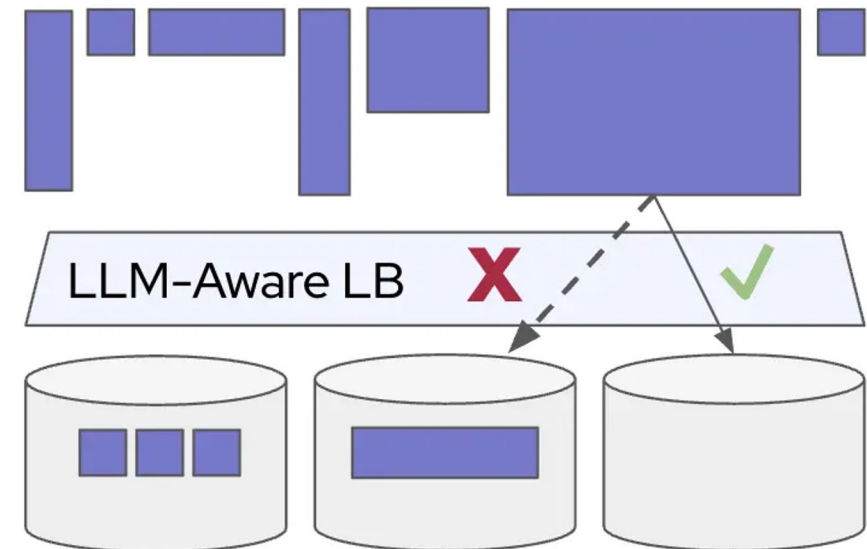
Fast, uniform, cheap



3-4 orders of magnitude more QPS

LLM requests:

Slow, non-uniform, really expensive



6-9 OoM more expensive per request

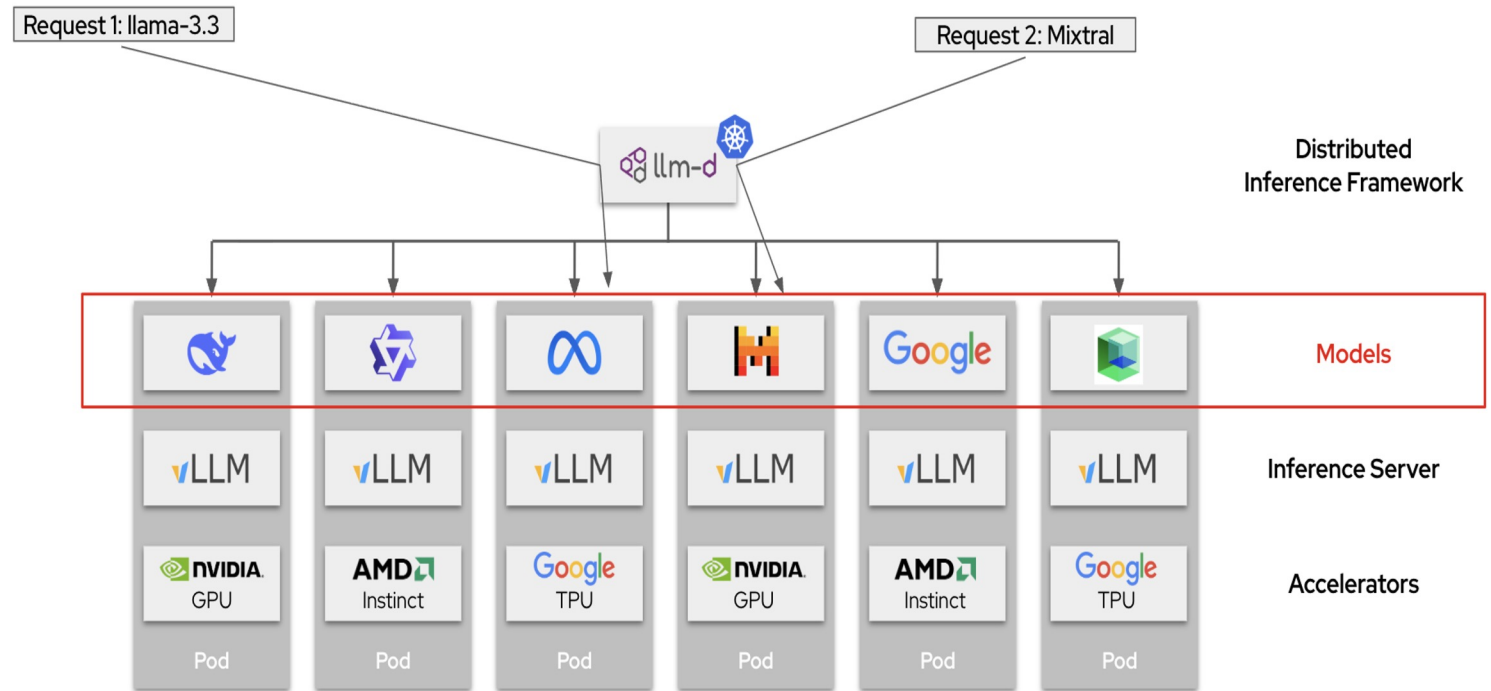


# What is llm-d?

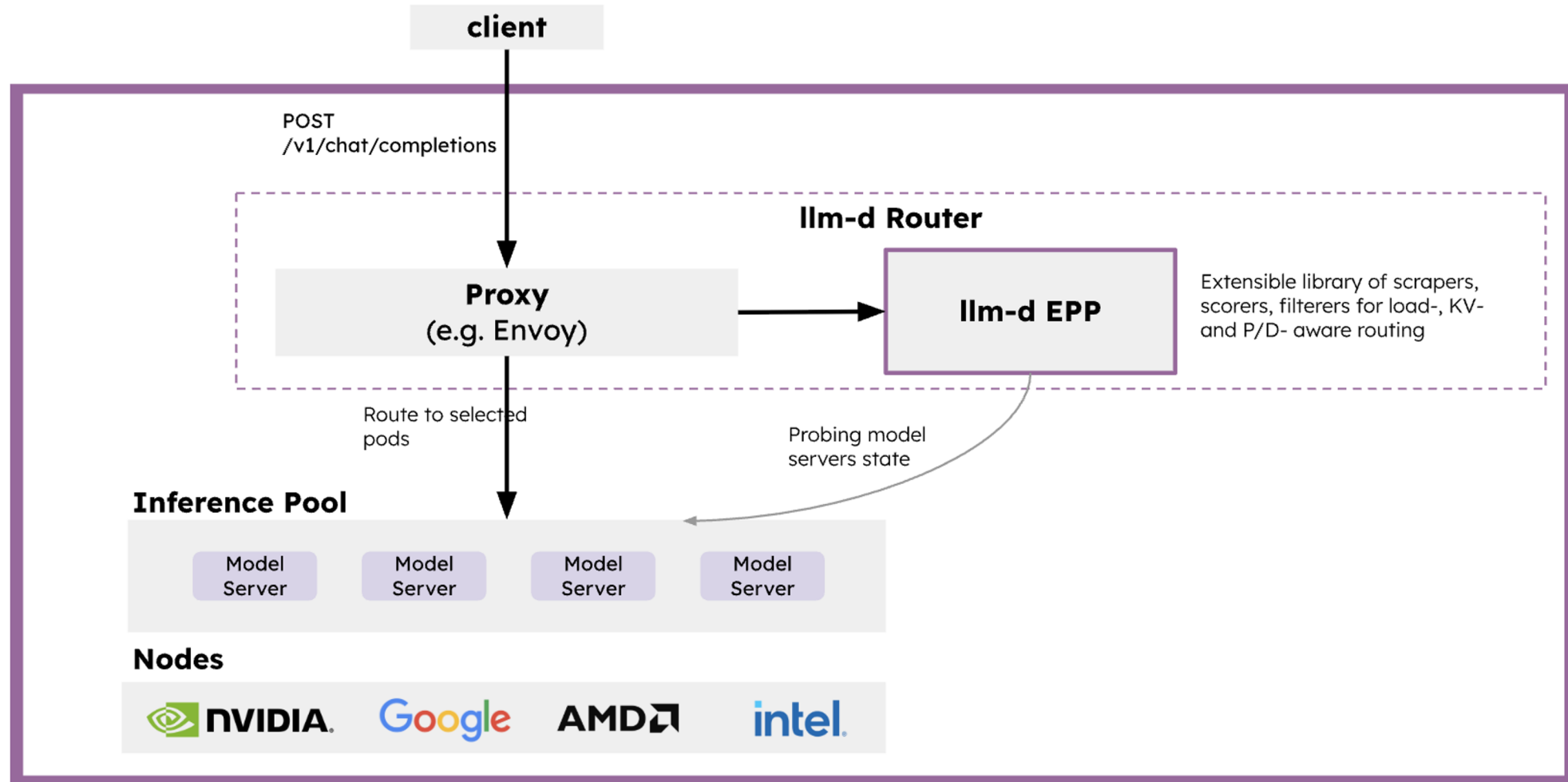
An open-source, Kubernetes-native framework for distributed LLM inference

An open-source framework for **distributed** large language model (LLM) inference that **runs natively on Kubernetes**.

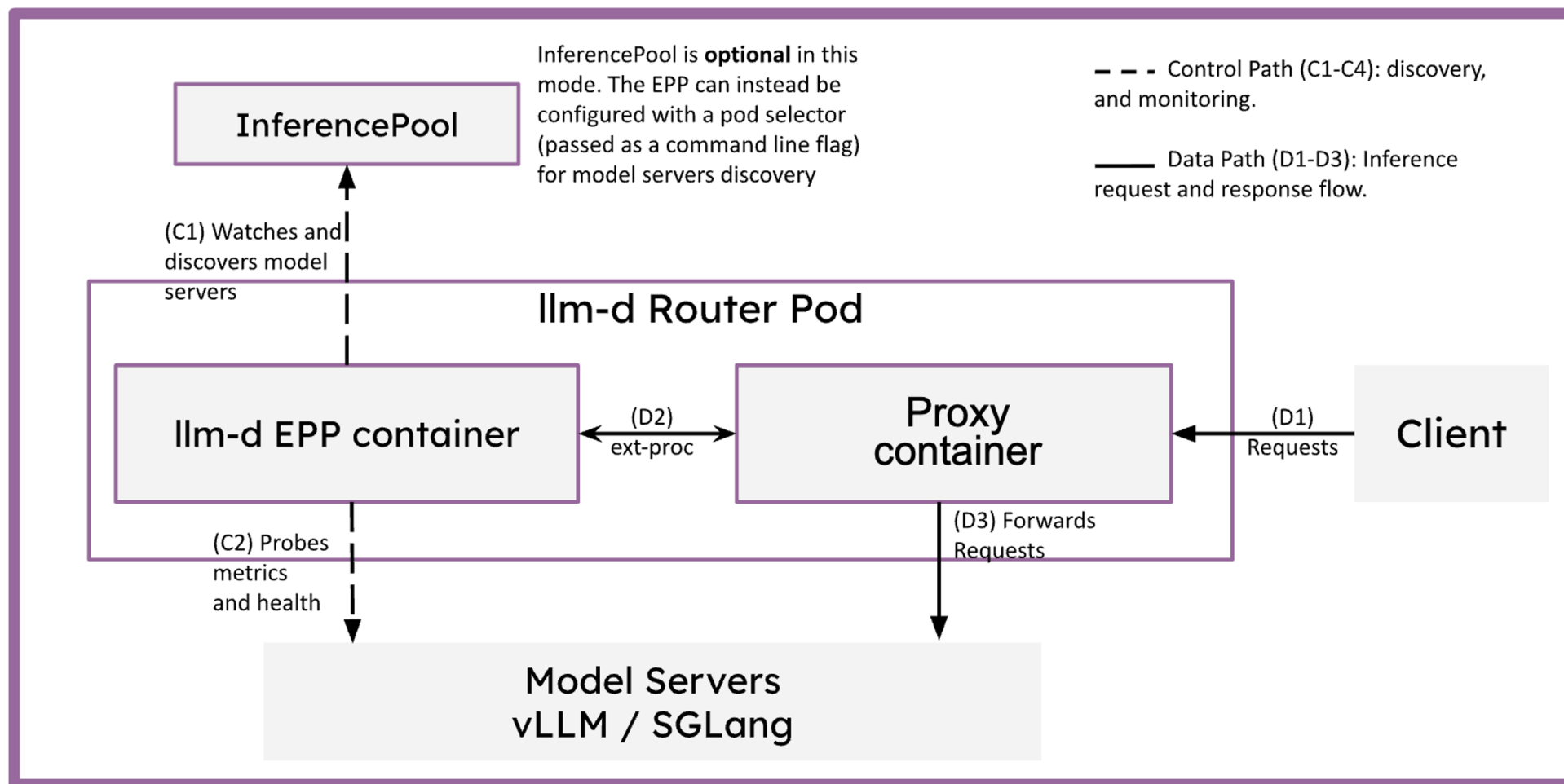
- ▶ Joint **open source** initiative by Red Hat, Google, Hugging Face, and many more organizations
- ▶ **Deployable anywhere** - k8s or non-k8s (new) + increasing hardware support
- ▶ **Distributed and pluggable architecture**
- ▶ **Guided deployment paths** showcasing **advanced scheduling techniques** and **serving patterns**



# Ilm-d Core Components: Router, Inference Pool, Model Servers

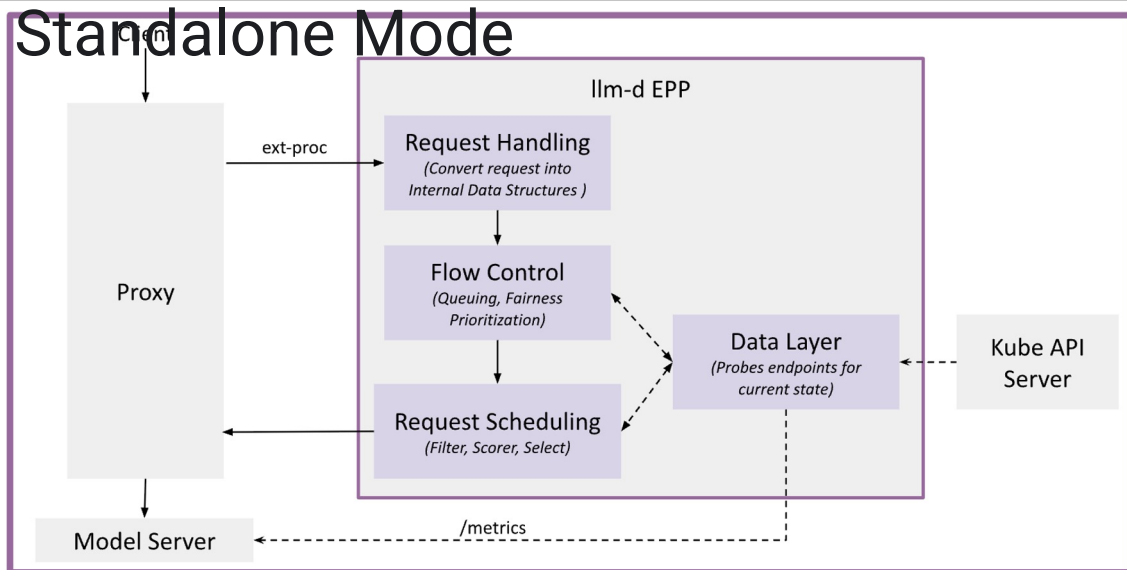


# Proxy

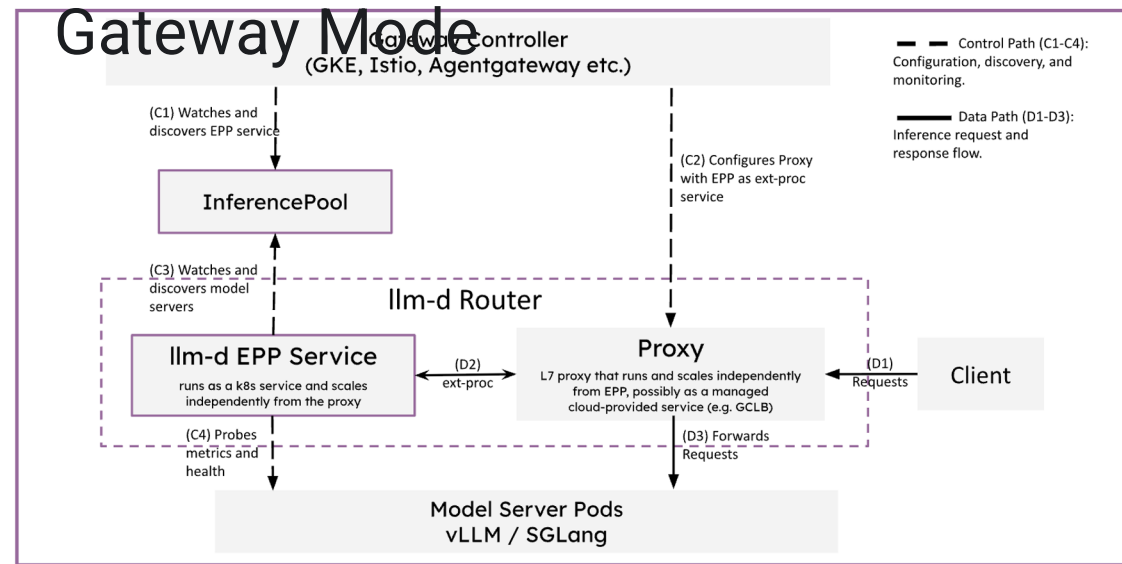


# Proxy

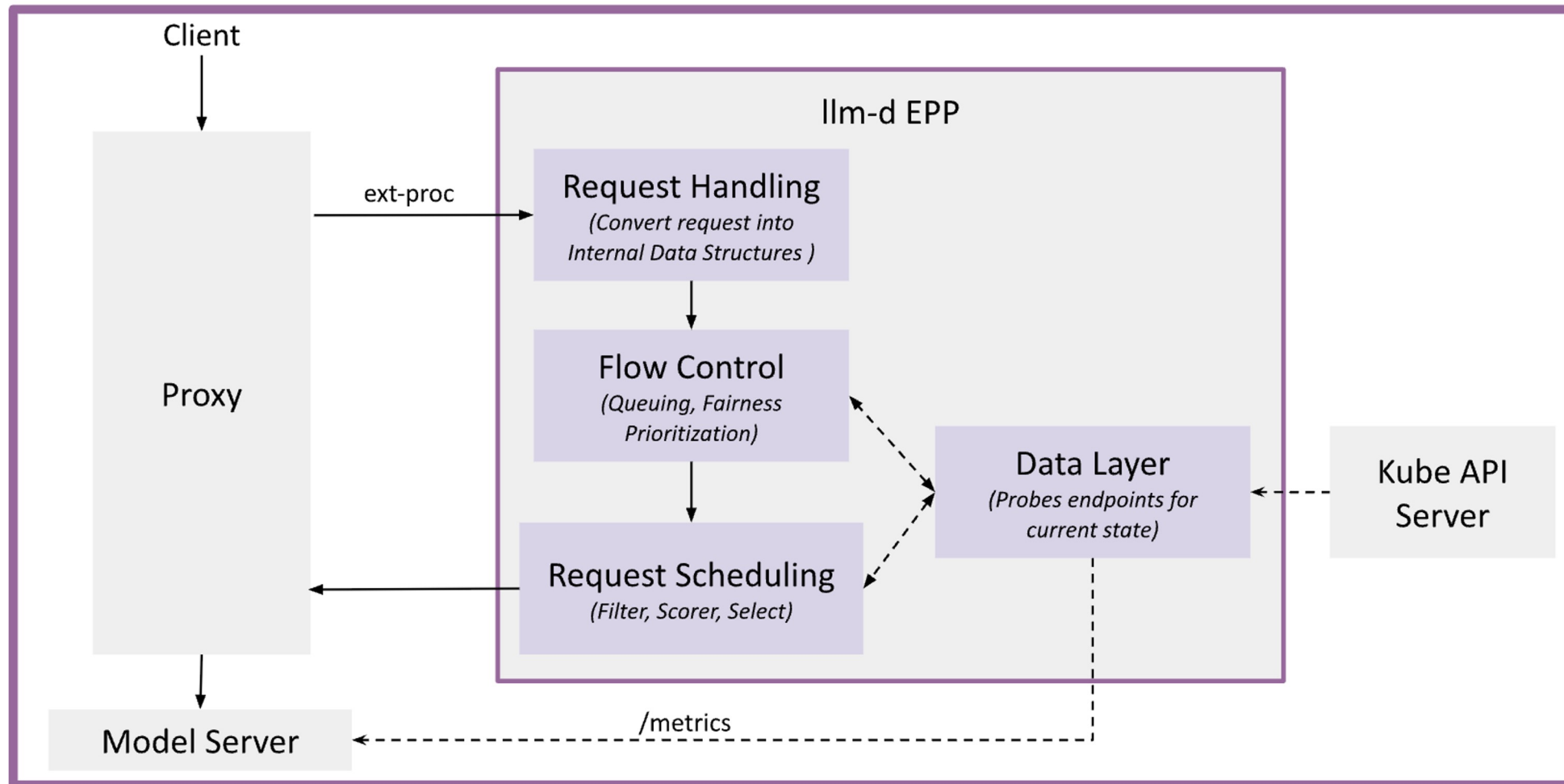
## Standalone Mode



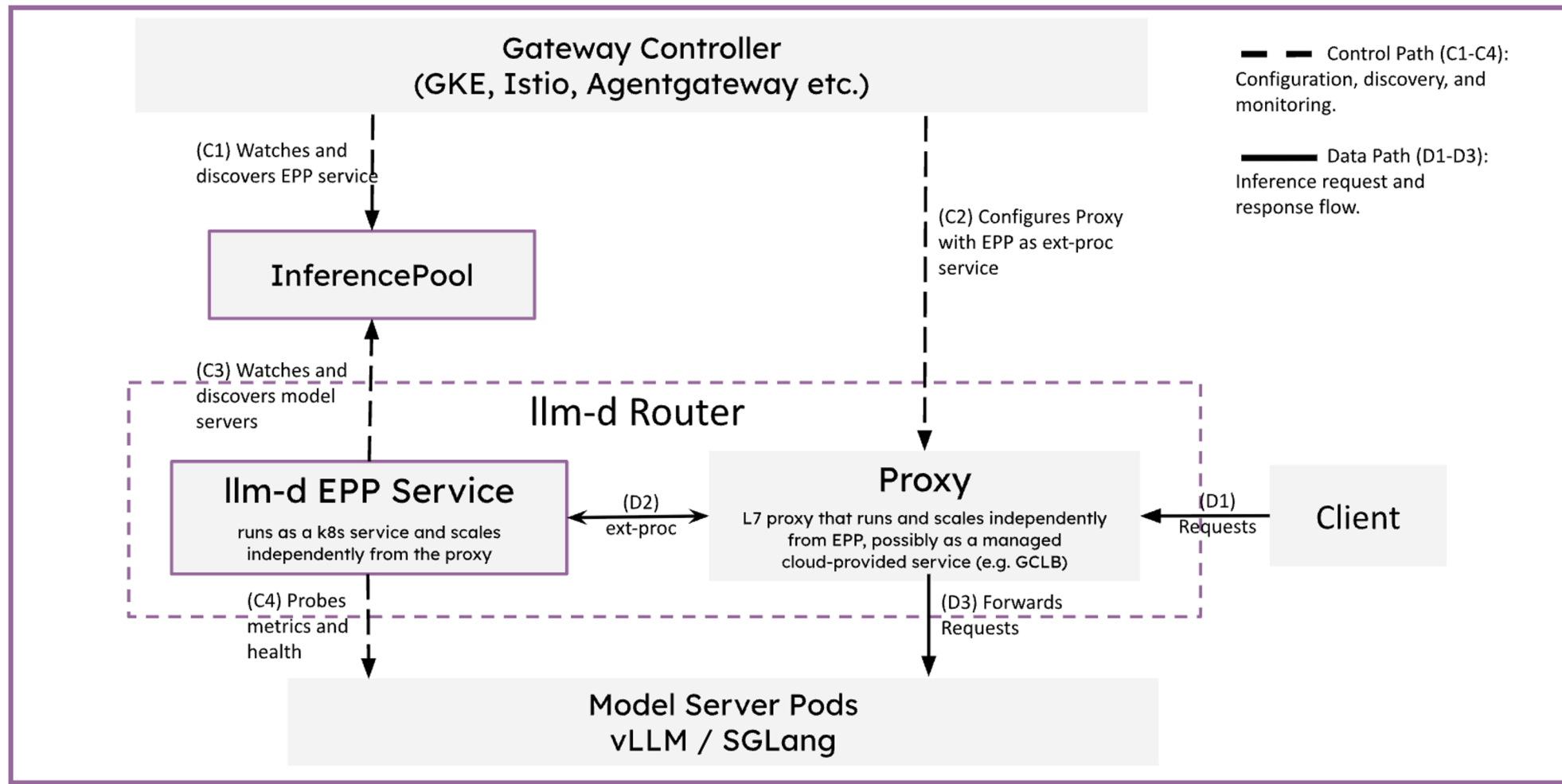
## Gateway Mode



# Endpoint Picker (EPP)



# InferencePool



# Model Servers

Key responsibilities:

- **Serve inference requests** via an OpenAI-compatible API (`/v1/completions`, `/v1/chat/completions`)
- **Expose metrics** (KV-cache utilization, queue depth, active requests) that the EPP uses for intelligent scheduling
- **Manage KV-cache** on GPU memory, including prefix caching for repeated prompt prefixes
- **Support parallelism strategies** such as Tensor Parallelism (TP), Data Parallelism (DP), and Expert Parallelism (EP) for large models



# Well-Lit Paths

Well-lit paths are curated, end-to-end guides for llm-d's native inference patterns, intelligent routing algorithms, and runtime traffic regulation engines.

These guides are organized into two native architectural groupings:

## Foundations

Individual functional features, sophisticated routing algorithms, and physical inference execution paths.

## Workloads

Cohesive, production-grade deployment guides that compose our capability building blocks into one horizontal stack tuned for a use case.

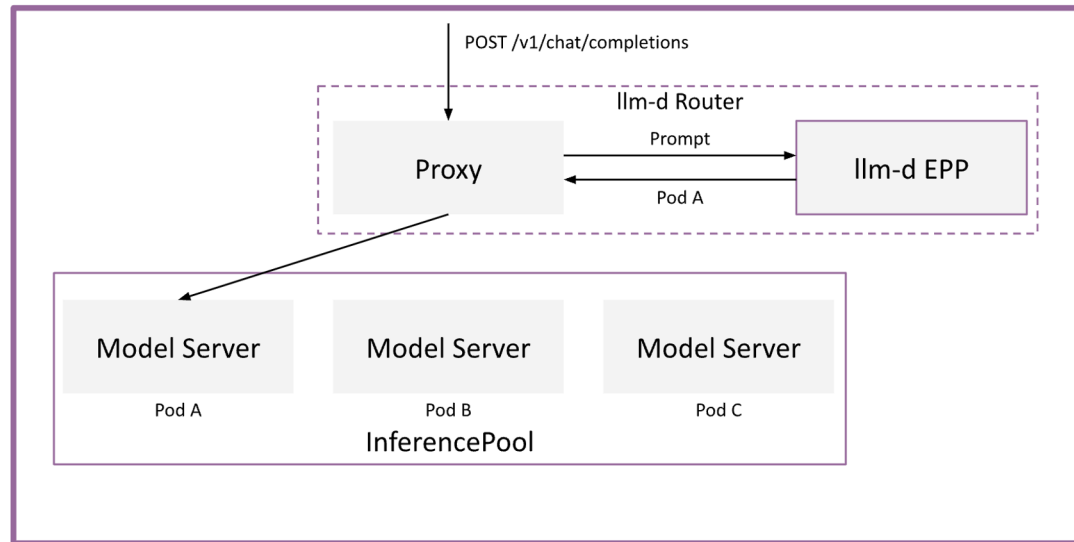


# Optimized Baseline

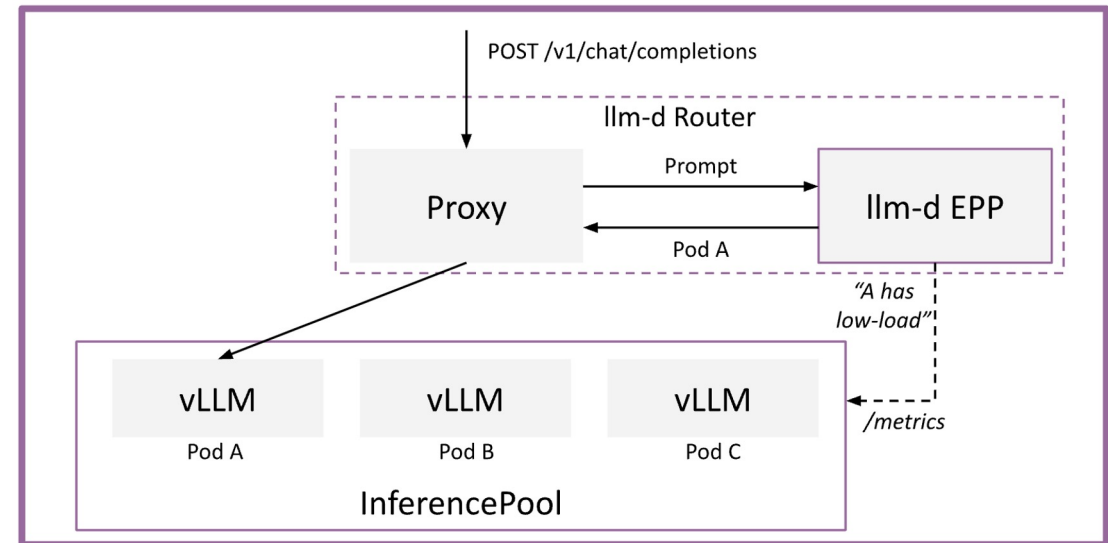
LLM requests break all three assumptions. They are:

- **Multi-turn** - conversations and agentic tool loops send the same growing prefix repeatedly
- **Slow** - a single request can take over a minute generating tokens
- **Non-uniform** - range from 1000s of reasoning tokens to a 100k+ context tokens

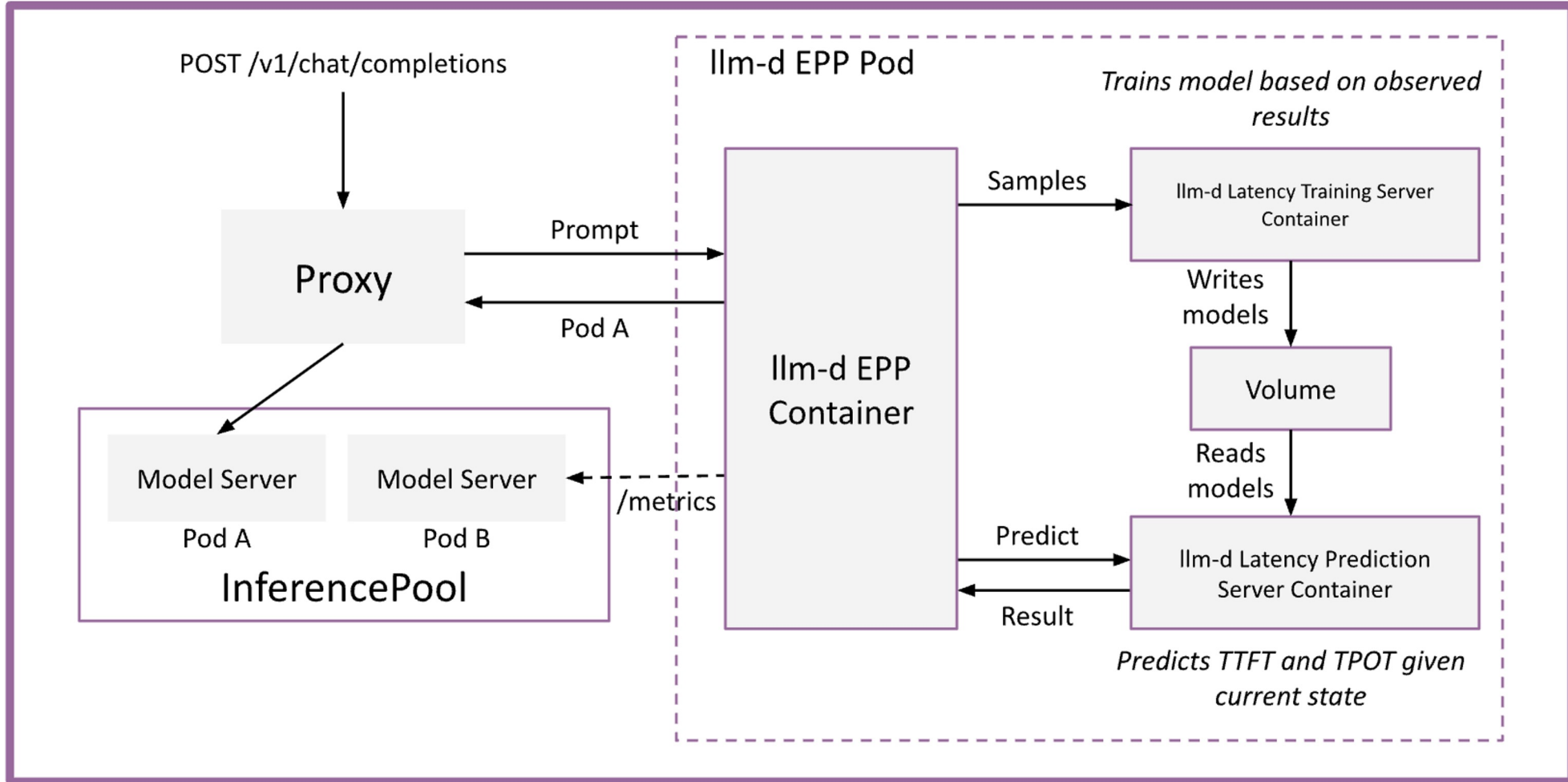
## Prefix-Aware Scheduling



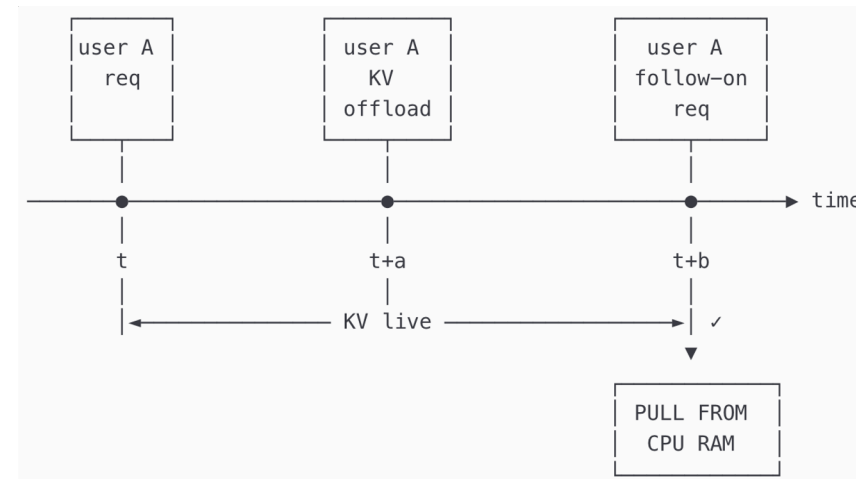
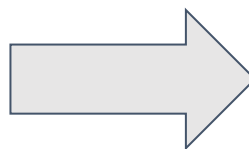
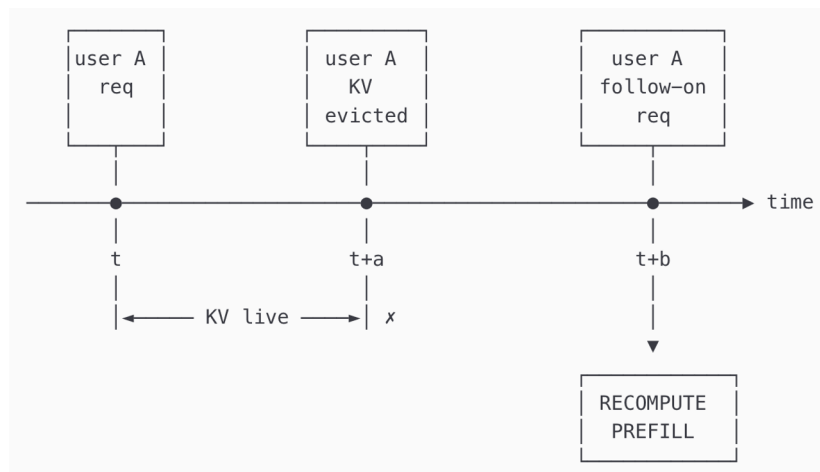
## Load-Aware Scheduling



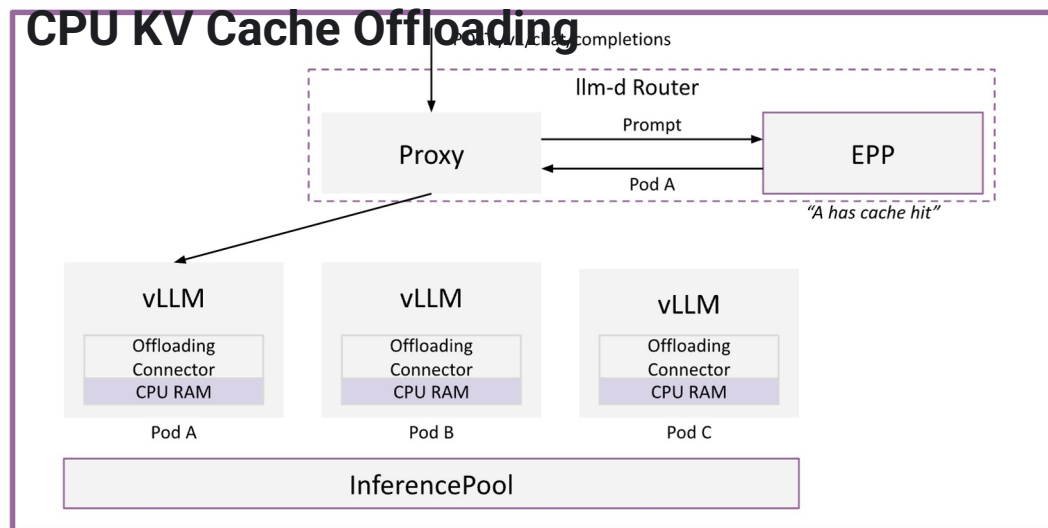
# Predicted Latency-Based Scheduling



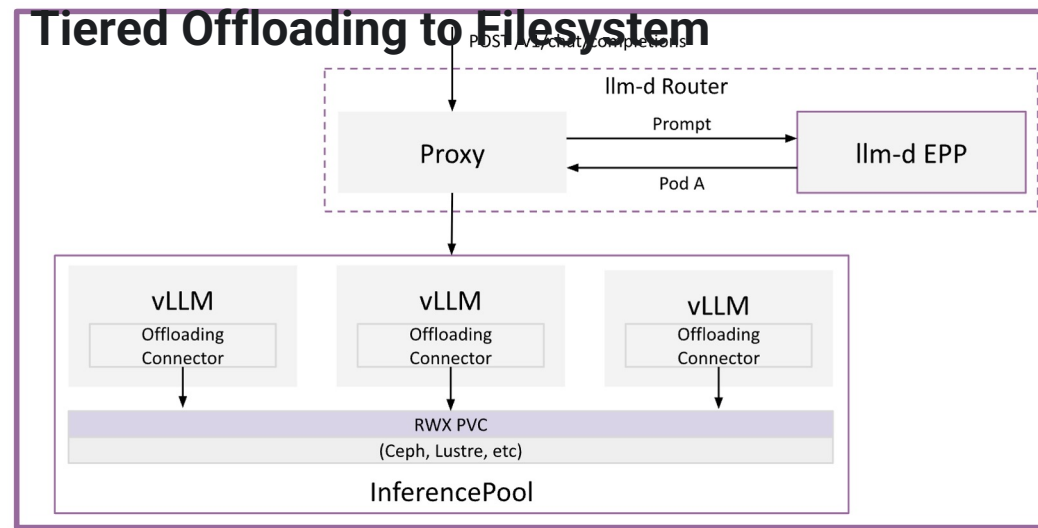
# KV Cache Offloading



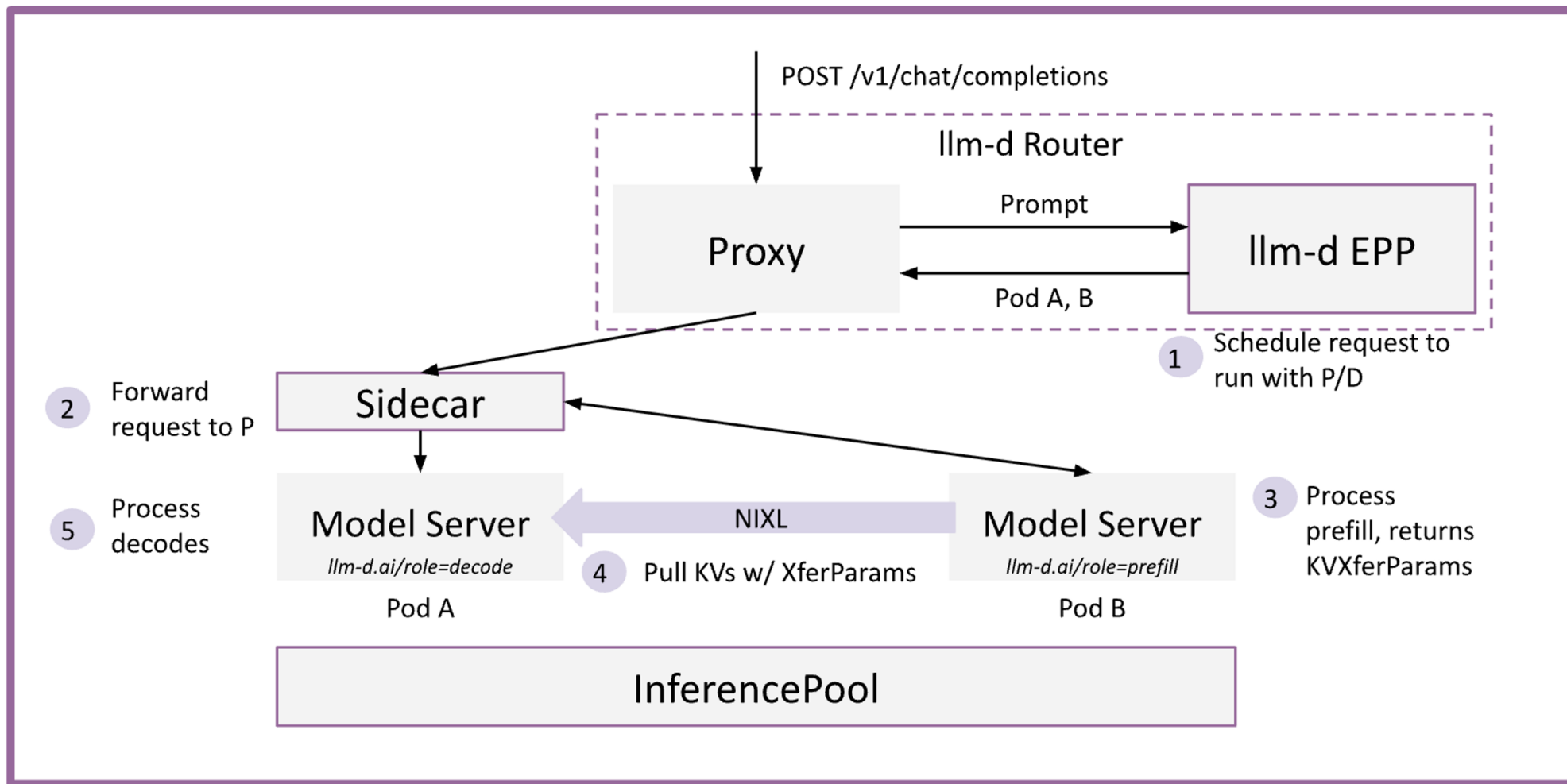
## CPU KV Cache Offloading



## Tiered Offloading to Filesystem

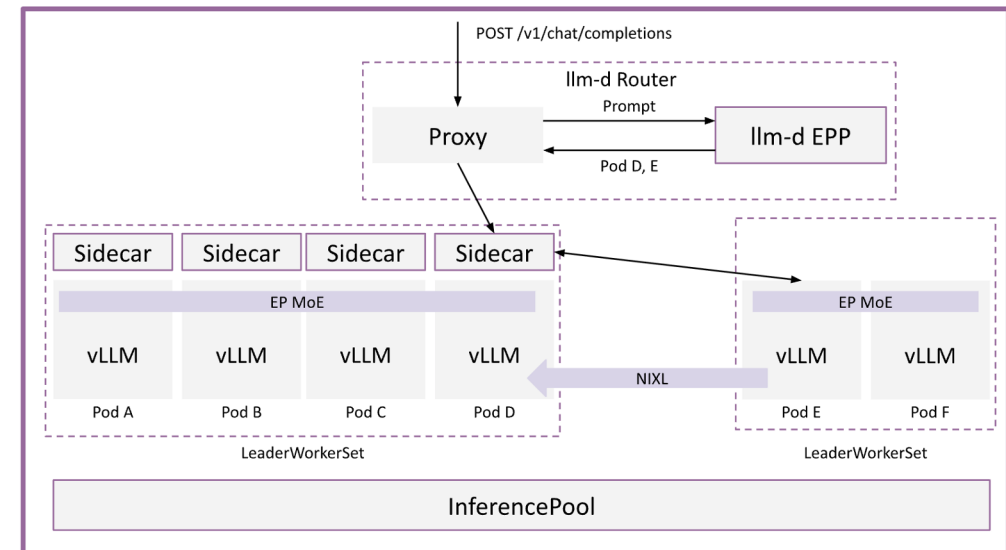
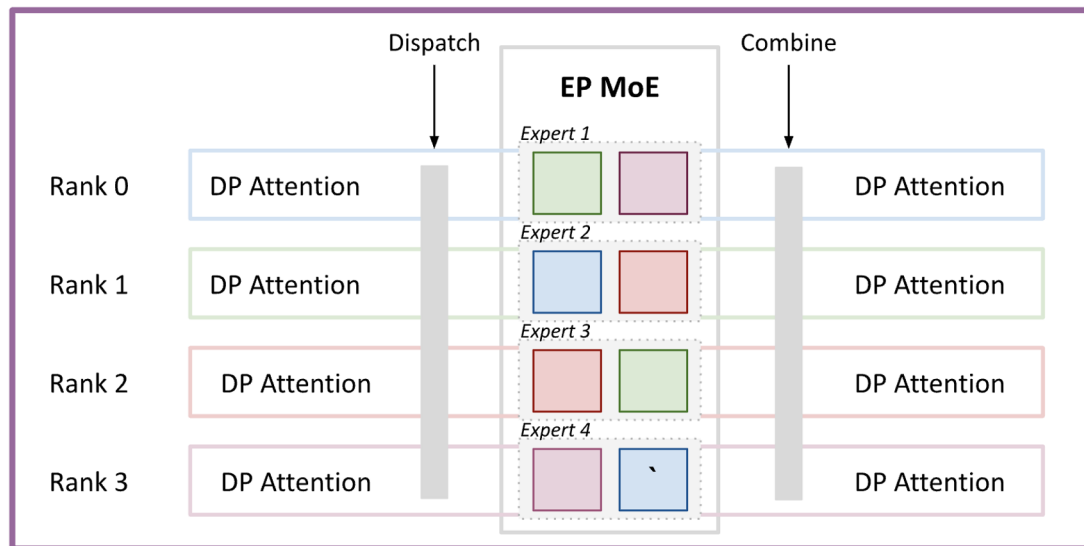


# P/D Disaggregation



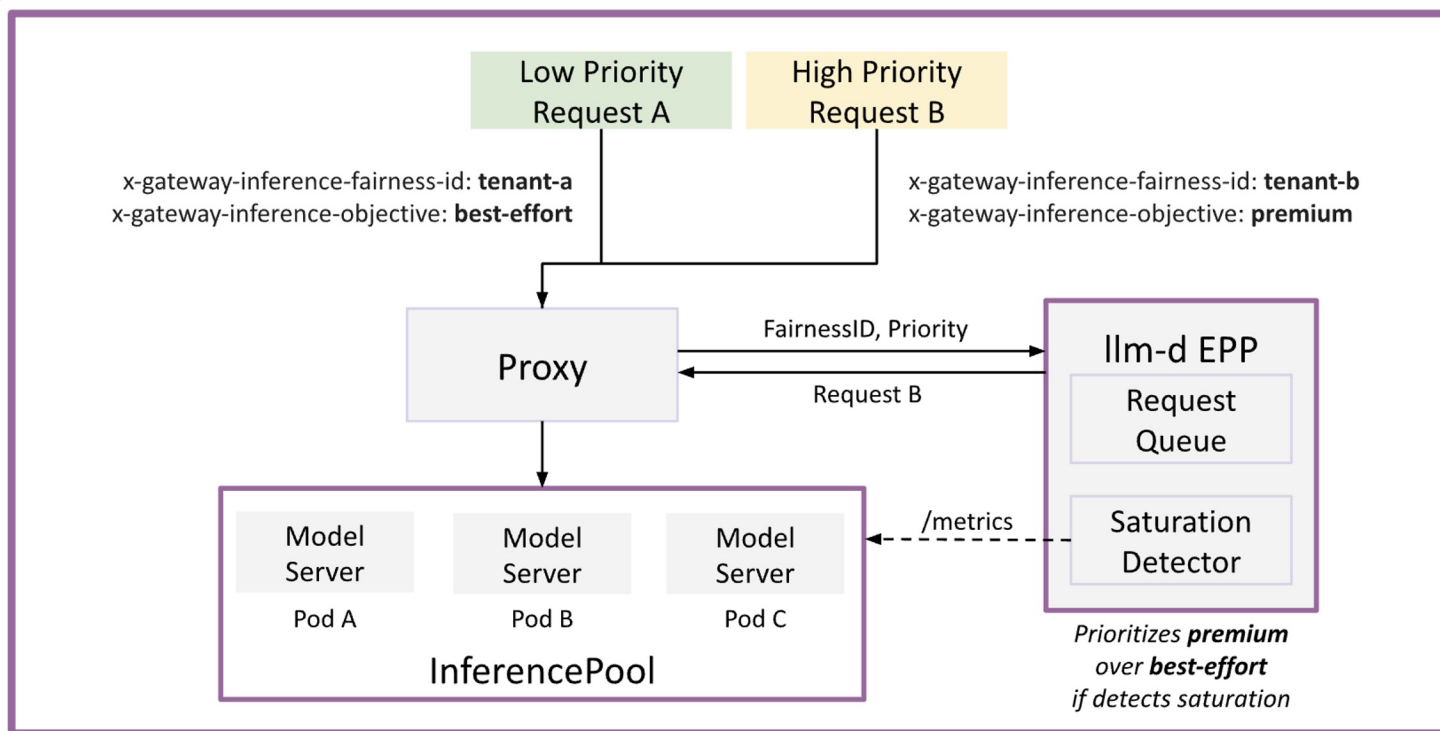
# Multi-Node Wide Expert Parallelism

- **Scales to multiple nodes** - since the collective operations (dispatch/combine) are sparse - tokens are only sent to the expert rank after routing - they consume much less bandwidth than the all-reduces used in TP setups, making them suitable to run over slower interconnects (IB, RoCE rather than NVLink)
- **Avoids KV replication** - since attention is data-parallel (TP=1 in every DP group), there is only one copy of each tokens' KV



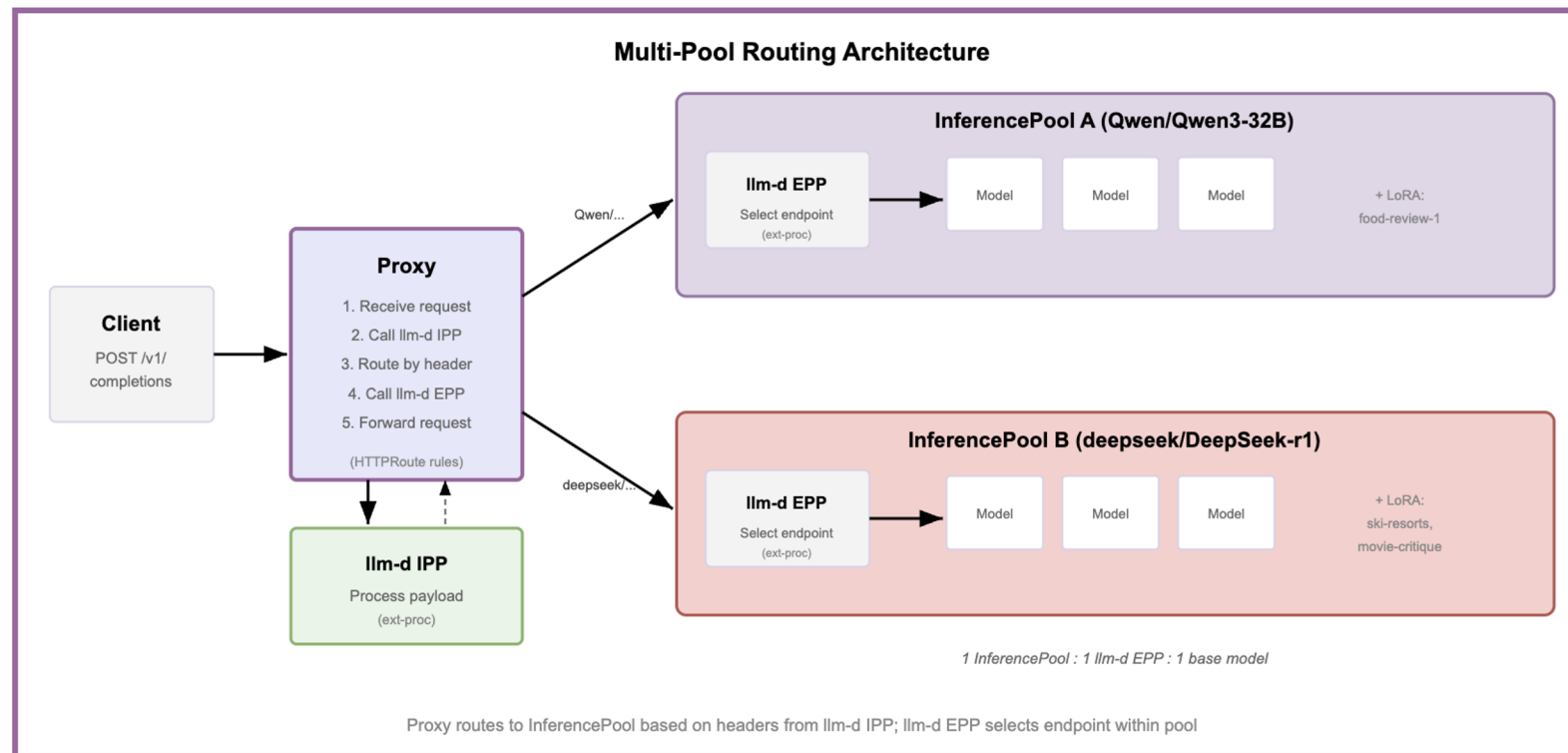
# Flow control

- Certain tenants are **higher-priority** than others (e.g. paid vs unpaid)
- Certain requests have **different-SLOs** than others (e.g. batch vs online)
- Certain tenants are more active than others - we want **fairness** between them



# Multi-Model Routing

Traditional path-based routing cannot distinguish between these models when they share the same API path (`/v1/chat/completions`). The **Inference Payload Processor (IPP)** solves this by extracting the model name from the request body and routing to the appropriate InferencePool.



# Workload Autoscaling

Effective LLM autoscaling requires proactive, SLO-aware signals that reflect the true state of the inference system — queue depth, in-flight request counts, and KV cache pressure — so that capacity can be added before end-user latency is impacted.

The llm-d stack provides two primary paths for workload autoscaling. The EPP path uses an HPA created and managed by KEDA. The WVA path publishes desired-replica signals that can be consumed by an HPA or KEDA, depending on the deployment configuration.

## KEDA + EPP Metrics

This path configures KEDA's Prometheus scaler with signals emitted directly by the Endpoint Picker (EPP). By using metrics such as queue depth and running request counts, KEDA's generated HPA reacts to queueing and active request concurrency, then reduces replicas when demand falls.

- **Best for:** Deployments on homogeneous hardware where each target model-server pool can be isolated by metrics and scaled independently.
- **Signals:** EPP metrics (queue depth, running request count).
- **Components:** KEDA and Prometheus; Prometheus Adapter is not required.

## Workload Variant Autoscaler (WVA)

The Workload Variant Autoscaler (WVA) is designed for operators running multiple model variants on shared, potentially heterogeneous GPU hardware. It continuously monitors KV cache utilization, queue depth, and performance budgets to determine optimal replica counts across variants (e.g., A100 vs. L4).

- **Best for:** Multi-variant deployments where cost-aware capacity allocation is required.
- **Signals:** KV cache utilization, queue depth, energy/performance budgets.
- **Components:** WVA controller and KEDA. WVA publishes a `wva_desired_replicas` metric for any KEDA `ScaledObject` (or HPA) annotated with `llm-d.ai/managed: "true"`, and KEDA drives the scale from it. The `VariantAutoscaling` CRD is deprecated as of llm-d 0.8.0.

# Fast Model Actuation

Fast Model Actuation (FMA) attacks vLLM startup time with two complementary techniques: **vLLM sleep/wake**, which parks a loaded model's tensors in main memory and restores them to the GPU in seconds — skipping model loading and CUDA-graph compilation entirely — and a **launcher** process that loads the vLLM Python modules once and spawns child vLLM instances on demand, skipping that module import. Together they turn model swap-in and replica scale-up from a minutes-scale cold start into a near-instant operation — without changing steady-state serving performance.

## Architecture

FMA circumvents that exclusive, lifetime-long GPU allocation with a **dual-pods** design coordinated by the FMA controllers:

- **Server-requesting pods** reserve GPU resources through the Kubernetes scheduler and kubelet — keeping cluster capacity accounting accurate — but do not run inference themselves.
- **Launcher pods** (server-providing) run vLLM while requesting **zero** GPUs. They can access all of their node's GPUs through provisions that don't count as scheduler usage, and the FMA controllers point vLLM at the specific GPU(s) reserved by the requesting pod via `CUDA_VISIBLE_DEVICES`.
- **FMA controllers** create and delete launchers, bind requesters to launchers and to the vLLM instances within them, create and delete those vLLM instances, and orchestrate sleep/wake so a fixed GPU pool can host many model variants.

## Actuation paths

When capacity is needed, the controller selects a launcher and takes the fastest available path:

- **Hot start** — wake an already-resident, sleeping instance by moving weights from CPU memory back to the GPU over PCIe (sub-second).
- **Warm start** — an existing launcher creates a fresh vLLM instance using preloaded modules (tens of seconds).
- **Cold start (with launcher)** — create a new launcher pod, then initialize the instance; still faster than a no-FMA cold start, which additionally pays pod scheduling and image pull.

To bound memory, the controller keeps at most a configurable number of instances sleeping per accelerator, evicting the least-recently-used instances when that budget is exceeded.



# Serve different types of workloads

## Agentic Workloads

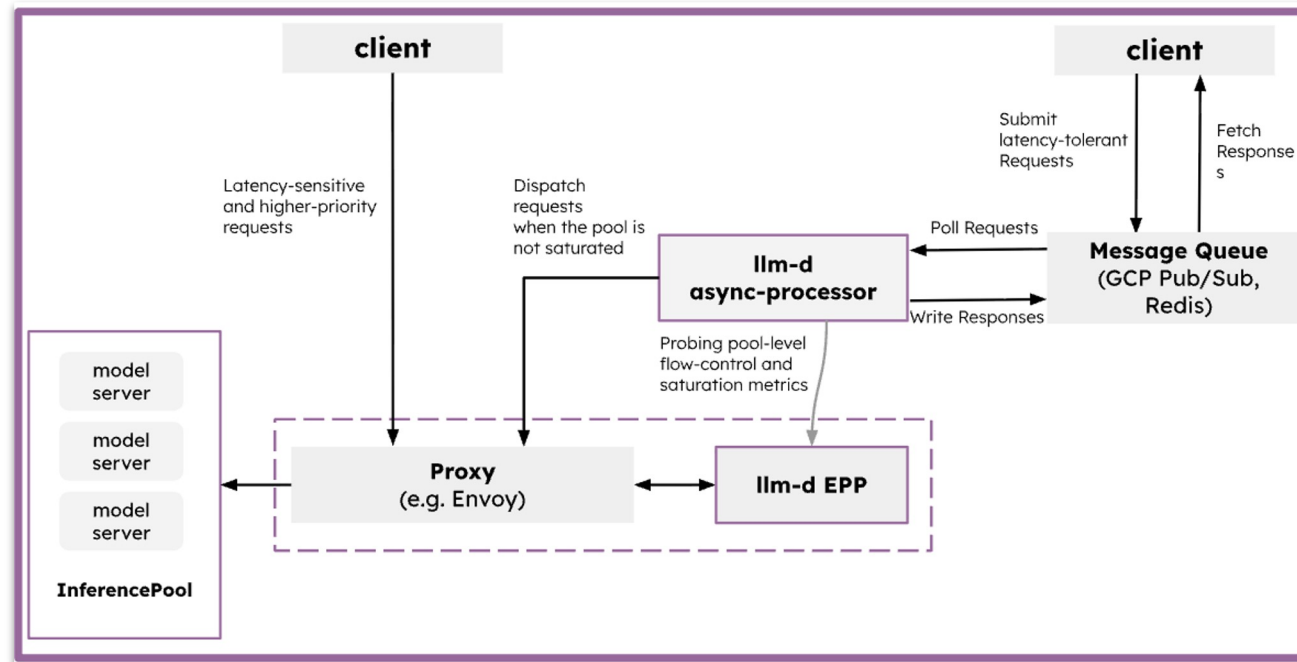
- **Massive context reuse** — branches of a reasoning tree, turns of a tool loop, and children of a swarm share most of their context (system prompts, tool definitions, conversation so far).
- **Program-level objectives** — users care about whole-program completion time, not the latency of any single call; one stalled branch can hold up the entire program.
- **Typed, predictable lifecycles** — durable system prompts and tools versus ephemeral scratchpad thoughts, with predictable flow the stack could anticipate but instead reacts to.

## Multimodal Workloads

- **Images** — When an image is embedded inline (raw bytes) in the request, EPP reads its width and height directly from the image content. When an image is supplied as a URL, EPP does **not** fetch the remote content and falls back to its configured defaults.
- **Video** — EPP cannot inspect video content directly, so clients should supply video metadata via request headers. Any header that is omitted falls back to the corresponding EPP default:
  - `x-llm-d-video-fps` — the video's frames per second.
  - `x-llm-d-video-duration-seconds` — the video's length in seconds.
  - `x-llm-d-video-resolution` — the video frame resolution as `WIDTHxHEIGHT` (e.g. `1280x720`).

## Batch Workloads

- **Batch Gateway**: An enterprise-grade, fully managed **OpenAI-compatible Batch API** (`/v1/batches`, `/v1/files`). Best for multi-tenant environments where clients require formal asynchronous job submission, file storage, status tracking, and strict separation of interactive vs. batch compute.
- **Asynchronous Processing**: A lightweight, low-overhead queue dispatch mechanism (using Redis or GCP Pub/Sub). Best for internal microservice architectures that require low-complexity background task processing or for filling "slack" capacity in your inference pool via dynamic dispatch gating.



# Ilm-d v0.8: From Platform to Control Plane

## I. Introduction — The Three Axes of v0.8

- Context: v0.7 made features deployable; v0.8 is the payoff
- Three defining themes:
  1. Graduating v0.7 experimental features to production
  2. Extending beyond Kubernetes (RL training, Slurm)
  3. Repositioning Ilm-d's identity: an inference control plane, not an engine fork
- Broadened scope: interactive chat, batch, multi-modal, and agentic workflows in one deployment

## II. Delivering on Promises: Feature Graduation

- Flow Control → production: FlowKey-based classification, per-flow queues, priority bands, tenant fairness, no-regret scheduling under saturation
- Batch Gateway → production: OpenAI-compatible `/v1/batches` + `/v1/files`, up to 50K requests/job, pluggable storage (PostgreSQL/Redis/S3), coexists with interactive traffic via Flow Control backpressure
- Multi-modal serving → production guides (LLaVA, Qwen-VL), including disaggregated P/D paths; simulator supports multi-modal payloads

# llm-d v0.8: From Platform to Control Plane

## III. Upstream, Not Fork: Aligning with vLLM

- Default images switch to unmodified upstream vLLM (`vllm/vllm-openai:v0.23.0`, ROCm, XPU); custom `llm-d-cuda-gb200/xpu/rocm` images removed
- Identity statement: llm-d's value is the control plane (router, endpoint picker, KV cache layer, autoscaler), not engine distribution
- Component renames reflect function: `llm-d-inference-scheduler` → `llm-d-router-endpoint-picker`; `routing-sidecar` → `router-disagg-sidecar`
- HPU (Intel Gaudi) support dropped (no upstream vLLM images)

## IV. Beyond Kubernetes: RL, Slurm, and File Discovery

- FileDiscovery plugin: filesystem-based endpoint discovery replaces K8s API dependency
- Primary driver: RL training loops — high-throughput rollout generation on Slurm/bare-metal, scheduler embedded in the training loop
- Core thesis: routing intelligence (prefix-cache locality, saturation awareness, flow control) is useful anywhere inference happens

## V. Intelligent Routing for the Agentic Era

- Agentic workload routing + Responses API: session affinity and prefix-cache state tracked across multi-step agent chains
- DP-aware scheduling with WideEP for DeepSeek-class MoE models (graduated, nightly CI)
- Predicted latency scheduling: full rewrite, production guide, CI validates real predictions vs. heuristic fallback



# llm-d v0.8: From Platform to Control Plane

## VI. Multi-Tier KV Cache and Hybrid Model Support

- HMA-aware KV offloading: hybrid architectures (full attention + sliding-window + Mamba/SSM + linear attention, e.g., Jamba); offload HBM → DRAM → storage respecting heterogeneous memory layout
- Mooncake connector: Alibaba's KV transfer engine as NIXL backend, joining UCX and UCCL (three transfer engines)
- Storage events: reactive tier management responding to capacity/eviction signals

## VII. New Well-Lit Paths and Ecosystem Reach

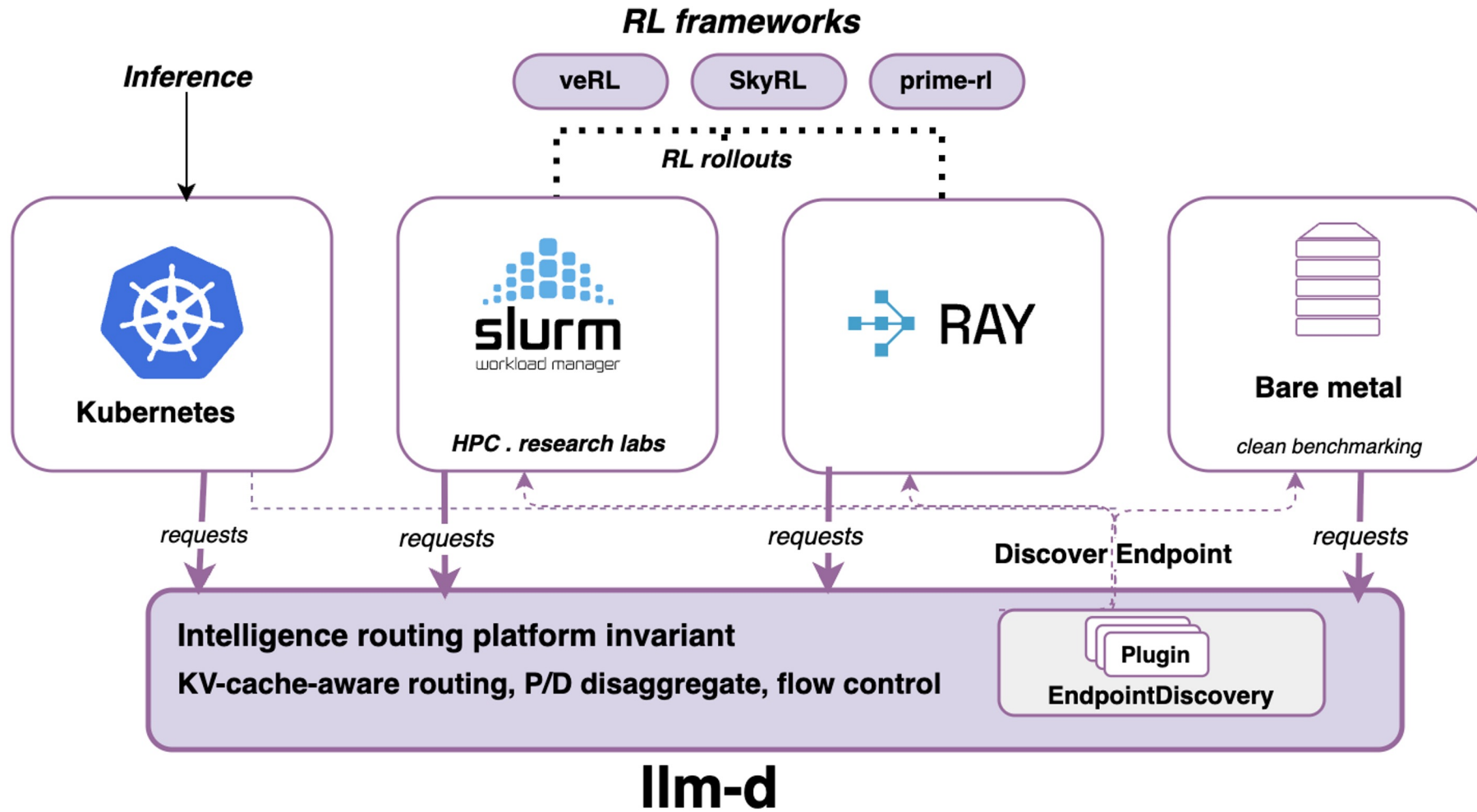
- DeepSeek-V4 on GB200 NVL72: dedicated guide with WideEP, NVSHMEM config
- Envoy AI Gateway guide: third gateway option alongside Istio and kgateway
- WVA autoscaler CRD migrates to `llm-d.ai` API group; infrastructure dependency updates (Istio 1.29.4, kgateway v2.3.3, GAIE v1.5.0)
- P/D disaggregation: GKE RDMA/DRA/DRANET configs + preflight network checks

## VIII. What This Means for You

- Platform teams: graduate Flow Control/Batch Gateway; simpler image pipeline; WVA migration is a breaking change
- ML researchers: cache-aware routing in Slurm RL loops without Kubernetes
- Agentic architects: session affinity, Responses API, engine-independent agentic routing

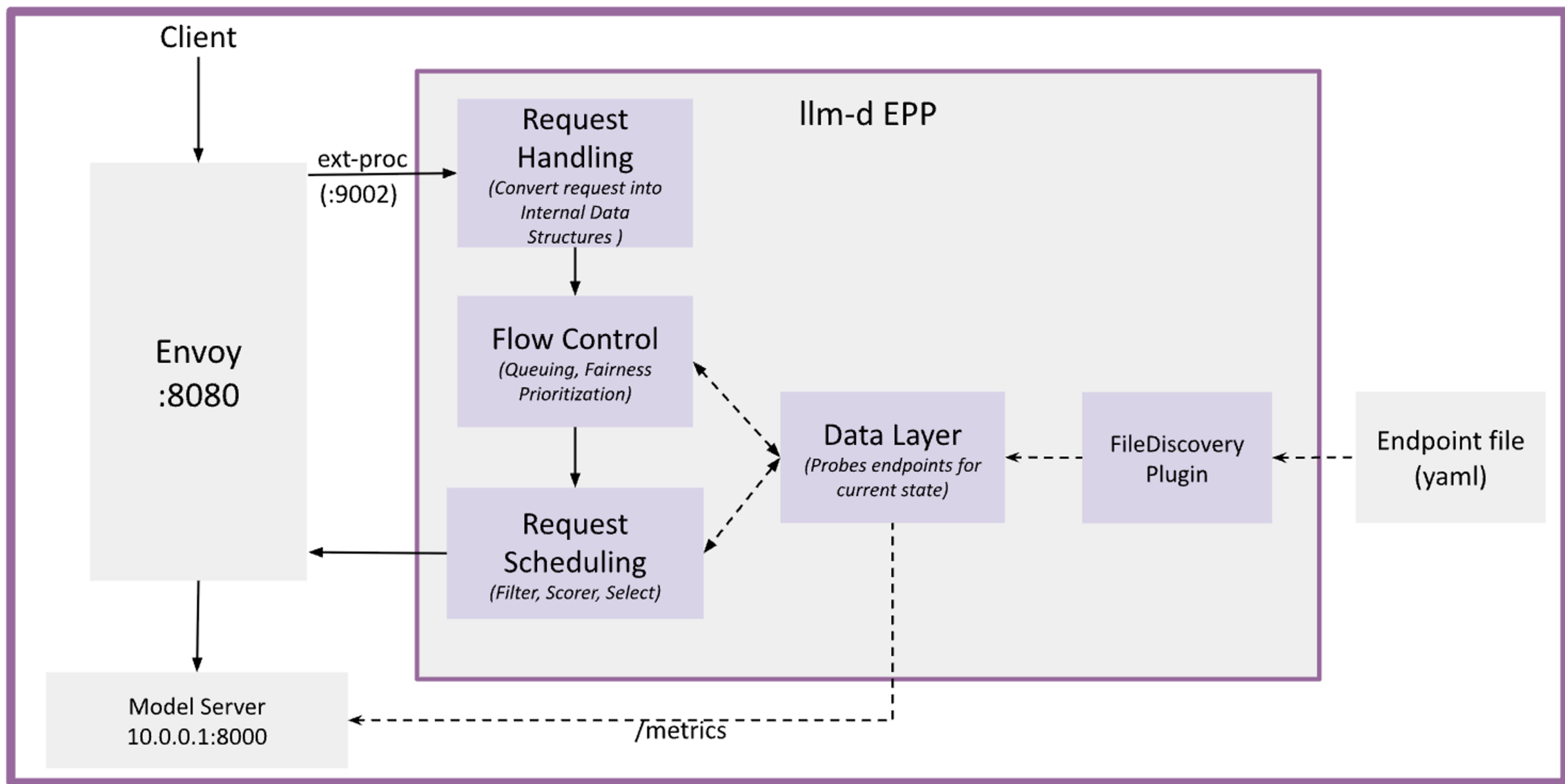


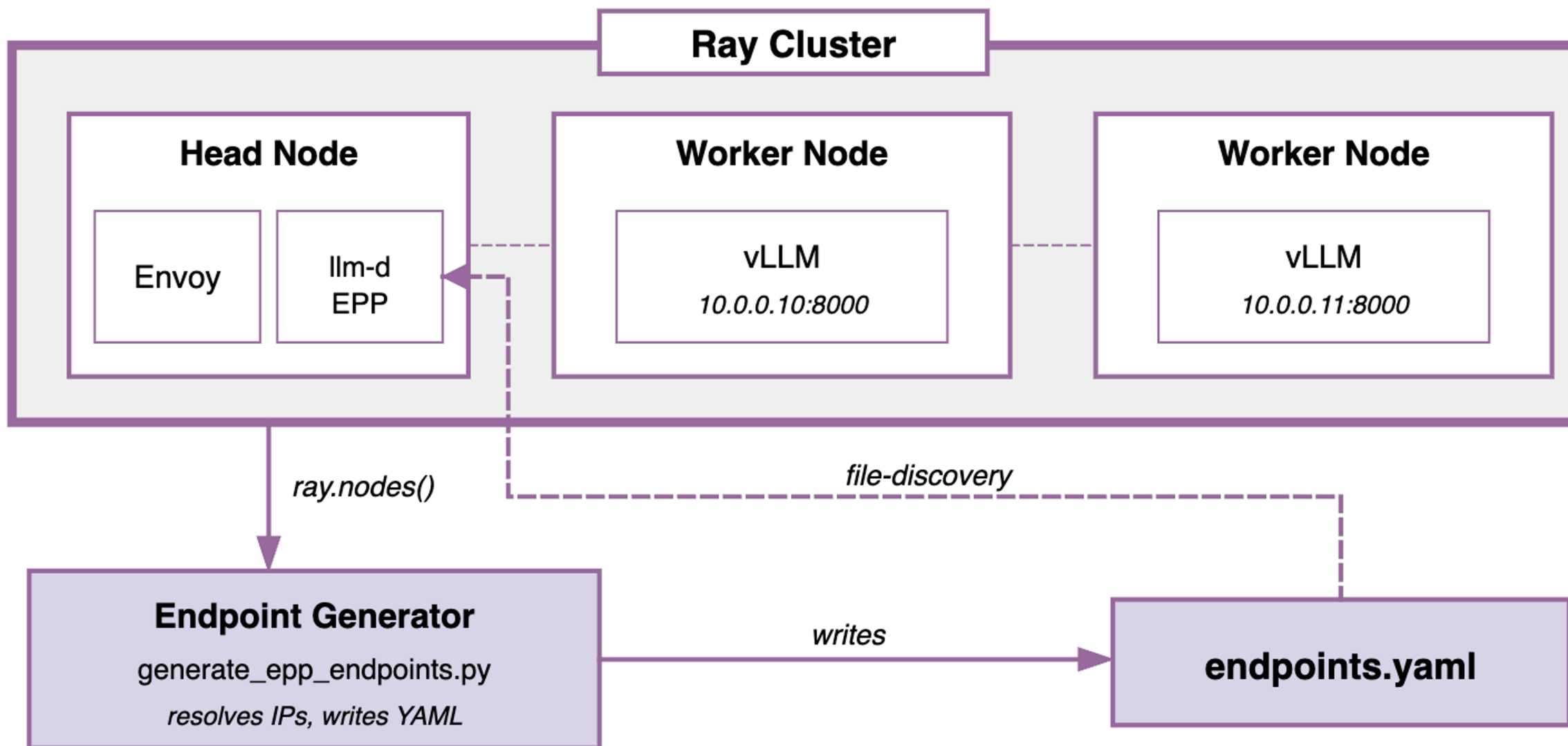
# Beyond Kubernetes: IIm-d Now Runs Anywhere



# The EndpointDiscovery interface

```
type EndpointDiscovery interface {  
    fwkplugin.Plugin  
  
    // Start begins discovery; blocks until ctx is cancelled or a fatal error occurs.  
    Start(ctx context.Context, notifier DiscoveryNotifier)error  
  
    // Ready is used to gate request-serving until the datastore is populated.  
    Ready()<-chanstruct{}  
}  
  
type DiscoveryNotifier interface {  
    // Upsert adds or updates an endpoint in the datastore.  
    Upsert(endpoint *EndpointMetadata)  
  
    // Delete removes an endpoint from the datastore.  
    Delete(id types.NamespacedName)  
}
```





# Ilm-d v0.9: Hardened for Scale

- Production-Hardened Router
- Disaggregation: Production-Grade Operations
- Routing Intelligence
- Multimodal and Diffusion Model Support
- KEDA-Native Autoscaling
- Reinforcement Learning: Incubation and Direction



Red Hat AI

LLM



# Keep building with us

<https://llm-d.ai/>

<https://llm-d.ai/slack>

<https://github.com/llm-d/>

微信公众号:



微信群:

